SWIFT SHIP TRACKER

A Low-Cost IoT-Based System for Real-Time Shipment Tracking and Condition Monitoring

A PROJECT REPORT

Submitted in partial fulfilment of the requirements for the project component of the academic curriculum

Submitted by:

Name	Register No
S. AMRITHA	**820723243002**
P. VISHAL	**820723243015**
S. SABTHAGIRI	**820723243013**
L. YOKESH	**820723243016**

Department: B.Tech Artificial Intelligence and Data Science

College Name: As-salam College of Engineering and Technology

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our project guide, [Guide Name], for the continuous support, patience, and valuable guidance extended throughout the duration of this project. Their insight into both logistics technology and low-cost embedded systems was instrumental in shaping the direction of this work.

We are also thankful to the Head of the Department, [HOD Name], and all the faculty members of the Department of Artificial Intelligence and Data Science for providing the laboratory facilities and encouragement needed to complete this project within a constrained budget.

We extend our appreciation to our families and friends for their constant encouragement, and to the local transport and courier operators who permitted the prototype trackers to be carried on their regular delivery routes, which made the real-world validation of this project possible.

Finally, we acknowledge the open-source hardware and software community, whose freely available libraries, documentation, and low-cost hardware designs made it possible to execute this project on a limited budget without compromising on core functionality.

S. AMRITHA, P. VISHAL, S. SABTHAGIRI, L. YOKESH

ABSTRACT

Modern supply chains depend on the ability to know where a consignment is, what condition it is in, and when it will arrive. Large logistics companies meet this need with proprietary tracking hardware and subscription platforms, but small courier firms, local transporters, small manufacturers, and student or community logistics groups are usually left with phone calls, handwritten registers, and customer enquiries that are answered only after a problem has already occurred. This project presents the design, development, and field testing of Swift Ship Tracker, a low-budget Internet of Things (IoT) shipment tracking system that provides near-real-time location and condition monitoring of consignments at a small fraction of the cost of commercial telematics products.

Each tracker node is built around a low-cost ESP32 microcontroller and combines a NEO-6M GPS receiver for position, a SIM800L GSM/GPRS module for cellular connectivity, an MPU6050 accelerometer for motion and shock detection, a DHT22 sensor for cargo temperature and humidity, and a reed-switch based door or lid sensor for tamper detection. Readings are published at an adaptive interval to a cloud database, where they are visualised on a live map dashboard, compared against rules such as geofence entry and exit, route deviation, excessive stoppage, temperature excursion, hard impact, and unauthorised opening, and forwarded as SMS or e-mail alerts to the responsible staff. A public tracking page allows a customer to follow a shipment using only a tracking ID.

The hardware cost per tracker was held to approximately Rs. 2,700 using off-the-shelf modules, a repurposed weatherproof enclosure, and a rechargeable 18650 cell with USB and optional solar charging. A prototype consisting of five trackers was carried on regular local delivery routes during a 14-day field trial, during which the system logged position and condition data continuously, correctly raised geofence, delay, shock, temperature and tamper alerts that were later confirmed against driver logs, and produced no false alerts on the control trips.

The results demonstrate that meaningful, continuous, data-driven shipment tracking is achievable using commodity hardware and free-tier cloud services, making the approach suitable for adoption by small logistics operators, educational institutions, and local enterprises. The report concludes with a discussion of limitations, lessons learned during deployment, and a roadmap for future work including LoRaWAN and NB-IoT connectivity, machine-learning based ETA prediction, and integration with e-commerce order systems.

Keywords: Shipment tracking, IoT, GPS, GSM/GPRS, ESP32, MQTT, geofencing, cold chain, low-cost telematics, supply chain visibility.

TABLE OF CONTENTS

CHAPTER 1: INTRODUCTION 4

CHAPTER 2: LITERATURE REVIEW 7

CHAPTER 3: SYSTEM ANALYSIS AND FEASIBILITY STUDY 9

CHAPTER 4: SYSTEM DESIGN AND ARCHITECTURE 12

CHAPTER 5: HARDWARE AND SOFTWARE REQUIREMENTS 16

CHAPTER 6: IMPLEMENTATION AND METHODOLOGY 19

CHAPTER 7: LOW-BUDGET COST ANALYSIS 22

CHAPTER 8: TESTING AND VALIDATION 24

CHAPTER 9: RESULTS AND DISCUSSION 26

CHAPTER 10: ADVANTAGES, LIMITATIONS AND APPLICATIONS 29

CHAPTER 11: FUTURE SCOPE 31

CONCLUSION 32

REFERENCES 33

APPENDIX A: SAMPLE SOURCE CODE 34

CHAPTER 1: INTRODUCTION

1.1 Background

Every product that a person buys, from a mobile phone to a packet of medicine, spends a part of its life in transit. Between the factory, the warehouse, and the customer's doorstep, a consignment may change hands several times and travel hundreds of kilometres on trucks, vans, trains, and two-wheelers. The reliability of this journey determines customer satisfaction, product quality, and ultimately the financial health of the business that is shipping the goods. As online shopping, home delivery, and regional trade continue to grow, customers have come to expect the same level of visibility over a parcel that they have over a ride-hailing vehicle: they want to see where it is, when it will arrive, and whether anything has gone wrong on the way.

Large courier and logistics companies have responded by investing in barcode scanning networks, vehicle telematics units, and proprietary tracking applications. These systems are effective, but they are built and priced for organisations that move millions of parcels. A small transporter with ten vehicles, a local courier firm serving a few towns, a rural cooperative moving perishable produce, or a college logistics club cannot justify the capital and subscription cost of such platforms. For these users, shipment visibility is still largely a matter of calling the driver and writing the answer in a notebook.

At the same time, the cost of the technology needed to build a tracker has fallen dramatically. Satellite navigation receivers, cellular modems, motion sensors, and Wi-Fi capable microcontrollers are now manufactured in very large volumes for the hobbyist and consumer market, and are available for a few hundred rupees each. Free cloud platforms and open-source mapping libraries make it possible to host a tracking dashboard without paying for servers. This project is motivated by the gap between what these inexpensive building blocks make possible and what small logistics operators actually use today.

1.2 Problem Statement

Existing shipment tracking in small and medium logistics operations is manual, delayed, and reactive. A dispatcher calls the driver, the driver describes his position from memory, and the information is entered into a register or spreadsheet. This approach suffers from several concrete problems:

- Location information is only as fresh as the last phone call, and it depends entirely on the driver's availability and honesty, so a consignment can be off-route or stationary for hours without anyone noticing.
- There is no record of the actual route, stoppages, or speed of a vehicle, so delays and disputes cannot be investigated objectively after the event.
- Condition-related incidents such as rough handling, an unauthorised opening of the cargo compartment, or a temperature excursion are not detected at all unless the receiver reports damage on delivery, by which time the cause cannot be established.
- Customers have no self-service way to check a shipment, which results in repeated enquiry calls that consume staff time.
- Commercial GPS trackers and fleet platforms exist, but they are priced with hardware margins and mandatory monthly subscriptions that are out of reach for many small

operators.

The central problem addressed by this project is therefore: how can a small logistics operator, institution, or community group track the location and basic condition of its consignments continuously and proactively, using hardware and service costs low enough to be feasible at scale, without depending on expensive proprietary telematics platforms?

1.3 Objectives of the Project

- To design a low-cost tracker node capable of measuring GPS position, speed, motion and shock, cargo temperature and humidity, and compartment opening for an individual consignment or vehicle.
- To build a wireless data pipeline that transmits tracker readings from the field to a cloud database over the public mobile network without any dedicated infrastructure at the vehicle or the warehouse.
- To develop a live web dashboard with a map view, route history, and per-shipment status that can be used by non-technical staff and, in a restricted form, by customers.
- To implement a rule-based alert layer for geofence entry or exit, route deviation, excessive stoppage, shock, temperature excursion, and tamper events, with notification by SMS and e-mail.
- To keep the total hardware cost per tracker within a strict low budget by using commodity modules, open-source software, and free-tier cloud services.
- To validate the system through a real-world field trial on regular delivery routes and compare the system's observations against driver logs and manual checks.

1.4 Scope of the Project

The scope of this project covers the design, prototyping, and field validation of a small-scale, five-tracker shipment tracking network. It includes the selection and bench calibration of low-cost sensors, firmware development for the microcontroller, cellular data transmission, cloud database and dashboard configuration, rule-based alerting, and a cost comparison with commercial alternatives.

The scope explicitly excludes certified or legally regulated tracking (for example, tachograph or AIS-140 compliant vehicle tracking), indoor positioning inside warehouses, integration with the internal systems of any specific courier company, and large-scale fleet deployment, which is identified as future work. The tracker is diagnostic and advisory: it reports and alerts, but it does not control the vehicle or the cargo.

1.5 Significance of Low-Budget Design

A recurring theme throughout this report is that every design decision, from the choice of microcontroller to the enclosure material and the cloud platform, was made with cost as a first-class constraint rather than an afterthought. This is a deliberate response to the reality that small operators run on thin margins, where a tracking solution costing several thousand rupees per vehicle plus a monthly fee is not viable regardless of its technical merit.

By targeting a per-tracker cost in the low thousands of rupees, built from modules that are already mass-produced for the maker community, this project demonstrates that

good-enough continuous tracking, sufficient to catch the most common and most costly problems in transit (a missing vehicle, a long unexplained stop, a deviation from the agreed route, rough handling, and a spoiled or tampered consignment), is achievable without the price tag of enterprise telematics platforms.

1.6 Organisation of the Report

Chapter 2 reviews existing literature and prior work in logistics visibility, satellite positioning, and low-cost IoT telemetry. Chapter 3 presents the system analysis, including a feasibility study across technical, economic, and operational dimensions. Chapter 4 describes the proposed system architecture and design diagrams. Chapter 5 lists the hardware and software requirements. Chapter 6 details the implementation methodology, including firmware logic and the deployment process. Chapter 7 presents the low-budget cost analysis and bill of materials. Chapter 8 covers testing and validation. Chapter 9 presents field-trial results and discussion. Chapter 10 discusses advantages, limitations, and applications. Chapter 11 outlines future scope, followed by the conclusion, references, and an appendix containing sample source code.

CHAPTER 2: LITERATURE REVIEW

This chapter surveys existing work relevant to the project across four overlapping domains: traditional logistics tracking practice, satellite-based positioning, low-cost cellular IoT telemetry, and condition monitoring of cargo. The purpose of this review is to establish the gap that the proposed system fills, namely an affordable, shipment-focused, continuous tracking solution positioned between manual phone-based tracking and expensive commercial telematics platforms.

2.1 Traditional Logistics Tracking Practice

Conventional shipment tracking is built around checkpoints. A parcel is scanned when it is accepted, when it enters or leaves a hub, and when it is delivered, and the status shown to the customer is simply the last scan recorded. Between two scans the parcel is effectively invisible. Small operators who do not have scanning infrastructure rely on the driver to report progress by phone or messaging application, and the information is recorded manually by a dispatcher.

This checkpoint model is adequate for long-haul parcel networks where the intervals between scans are measured in days, but it does not answer the questions that matter most to a local customer, such as how far away the delivery vehicle is right now, or whether the vehicle has been standing still for two hours. It also provides no information about the physical condition of the consignment while in transit.

2.2 Satellite Positioning and GPS Tracking

Global Navigation Satellite Systems (GNSS), of which the American GPS is the best known, allow a receiver to compute its position by measuring the arrival time of signals from several satellites. Consumer-grade receivers such as the u-blox NEO-6 family report position, speed, heading, and time in standard NMEA sentences over a serial interface, with typical open-sky horizontal accuracy of a few metres. Kaplan and Hegarty describe the principles of pseudo-range positioning and the factors that degrade accuracy, including poor satellite geometry, multipath reflection from buildings, and signal blockage under bridges and dense foliage.

Commercial vehicle trackers are essentially a GNSS receiver combined with a cellular modem and a small processor, and the underlying technology is identical to that available in hobbyist modules. The price difference between a commercial tracker and a self-assembled one therefore lies mainly in packaging, certification, support, and the recurring cost of the vendor's platform rather than in any fundamental technical capability.

2.3 Low-Cost Cellular IoT and Telemetry Protocols

The maker and open-hardware community has produced an extensive body of practical work using microcontrollers from the Arduino and ESP32 families combined with inexpensive GSM/GPRS modules such as the SIMCom SIM800 series to send data over the public mobile network. GPRS remains widely available in India and many other regions, uses very small data volumes for telemetry, and does not depend on any local Wi-Fi infrastructure, which makes it well suited to moving assets.

For the transport of readings, the Message Queuing Telemetry Transport (MQTT) protocol, standardised by OASIS, is a lightweight publish-subscribe protocol designed for constrained devices and unreliable networks. Plain HTTPS requests to a REST endpoint are an equally valid alternative for low message rates and are simpler to implement on a small device. Atzori, Iera and Morabito and, separately, Gubbi and colleagues survey the architectural elements of the Internet of Things and identify exactly this combination of sensing nodes, low-power connectivity, and cloud services as the dominant pattern for asset monitoring applications.

2.4 Cargo Condition Monitoring

Beyond position, the condition of the cargo is a significant concern for certain consignments. Cold-chain logistics for food and pharmaceuticals relies on temperature loggers to prove that a product stayed within its allowed range. Fragile and high-value goods call for shock or impact indicators, and security-sensitive goods call for seal or door-opening detection. Dedicated data loggers for these purposes exist, but they are commonly stand-alone devices that must be physically recovered and read after the journey, so that a fault is discovered only after delivery.

Low-cost MEMS accelerometers such as the MPU6050 can detect the sudden acceleration spikes associated with drops and collisions, temperature and humidity modules such as the DHT22 provide adequate indicative readings, and a magnetic reed switch can detect whether a lid or door has been opened. Combining these with the live telemetry link turns a passive logger into an active, alerting condition monitor.

2.5 Research Gap and Positioning of This Project

Reviewing the above bodies of work together reveals a clear gap. Manual tracking is cheap but delayed and unquantified; commercial telematics is continuous and quantitative but not cost-effective for small operators or for tracking individual consignments; and open-source IoT building blocks supply the low-cost components but have rarely been assembled and packaged specifically for shipment tracking with meaningful alert rules, a customer-facing tracking page, and a documented cost model.

This project positions itself at the intersection of these domains: applying low-cost GNSS, cellular, and MEMS sensing hardware to the specific use case of consignment tracking and condition monitoring, with a design philosophy that treats budget as a hard constraint rather than an optimisation target reached only after functionality is complete.

CHAPTER 3: SYSTEM ANALYSIS AND FEASIBILITY STUDY

3.1 Existing System

The existing system in most small logistics businesses consists of a manual follow-up workflow. A consignment is booked and handed to a driver, and the dispatcher records the vehicle, driver, and expected delivery time. If the customer or the dispatcher wants to know the status, the driver is called. Information about the position, stoppages, and condition of the goods exists only in the driver's memory and is recorded, if at all, after the delivery is complete.

This workflow has three structural weaknesses relevant to this project: the information is delayed, because it is available only when someone asks; the information is subjective and unverifiable, because it depends on what the driver reports; and there is no mechanism to detect a developing problem, such as a long unscheduled halt or a deviation from the route, until the customer complains.

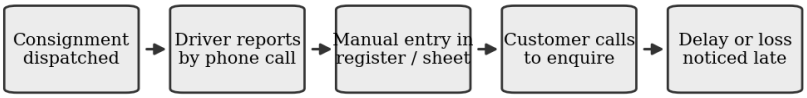


Fig 3.1: Existing manual tracking workflow: delayed, unverifiable, and reactive.

3.2 Proposed System

The proposed system introduces a continuous, low-cost tracker between the consignment and the logistics team. Each tracker records GPS position, speed, motion, shock, temperature, humidity, and compartment status at an interval that adapts to whether the vehicle is moving (for example, every 30 seconds) or parked (for example, every 5 minutes), and transmits the data over the mobile network to a cloud database. A rule-based layer compares each new reading with the configured rules and raises an alert when a condition is breached for a sustained period, rather than for a single noisy reading.

This shifts the dispatcher's workflow from calling drivers to watching a single map, and from discovering problems after delivery to responding to alerts while the shipment is still in transit. Customers are given a tracking page that shows the shipment's status and last known position, which removes most enquiry calls.

3.3 Comparison of Existing and Proposed System

Aspect	Existing (Manual) System	Proposed (IoT) System
Location update	On phone call only	Continuous (30 s to 5 min, adaptive)
Data type	Verbal, subjective	Quantitative GPS and sensor readings
Problem detection	After customer complaint	During transit, via rule-based alerts

Aspect	Existing (Manual) System	Proposed (IoT) System
Route record	None	Full route history stored in cloud
Condition data	None	Shock, temperature, humidity, door status
Customer visibility	Phone enquiry	Self-service tracking page
Cost per shipment	Recurring staff time	One-time low hardware cost, near-zero running cost
Scalability	Limited by staff headcount	Limited by hardware budget, easily scaled

Table 3.1: Comparison of the existing and proposed systems

3.4 Feasibility Study

3.4.1 Technical Feasibility

All hardware components required, namely the ESP32 microcontroller module, NEO-6M GPS receiver, SIM800L GSM module, MPU6050 accelerometer, DHT22 sensor, and reed switch, are commodity items with mature open-source libraries and extensive community documentation. GPRS coverage is available along the majority of the roads on which the trial was run, and the cloud database and dashboard platforms used offer free tiers sufficient for a pilot of this scale. The technical risk is therefore assessed as low, with the main risks being GPS signal loss in dense urban areas and mobile network dead zones, both of which are handled by buffering readings on the device and sending them when the connection returns.

3.4.2 Economic Feasibility

The target per-tracker hardware cost (detailed in Chapter 7) is low enough that even a modest operator can fund a pilot of several dozen units. Running costs are limited to a low-volume mobile data plan, and the free tiers of the cloud services remove subscription costs during the pilot phase. The economic case is strengthened by the value of early problem detection: a single recovered consignment, a single avoided dispute over a delay, or a single spoiled batch of perishable goods that is caught in time can outweigh the cost of several trackers.

3.4.3 Operational Feasibility

The system is designed to fit into, rather than replace, the existing dispatch workflow. The dashboard adds visibility on top of the same dispatchers and drivers. Preparing a tracker for a shipment requires only switching it on and registering its ID against the consignment, and routine maintenance consists of recharging the battery and checking the antenna connections, which requires basic training and not specialised technical staff.

3.5 Requirement Analysis

3.5.1 Functional Requirements

- The system shall record GPS position, speed, motion, shock, temperature, humidity, and compartment status at a configurable interval.
- The system shall transmit recorded data wirelessly to a cloud database over the mobile network.

- The system shall display live position and historical route for each tracker on a web-accessible map dashboard.
- The system shall compare each reading with defined rules and generate an alert when a rule is breached for a sustained duration.
- The system shall notify designated staff of alerts by SMS and/or e-mail.
- The system shall allow a tracker to be registered against a shipment with basic metadata (consignment ID, origin, destination, expected time of arrival).
- The system shall provide a public tracking page that shows the status and last known position of a shipment given its tracking ID.

3.5.2 Non-Functional Requirements

- Cost: total hardware cost per tracker shall not exceed the defined low-budget target.
- Power autonomy: each tracker shall operate for at least 48 hours on its battery without recharging under normal duty.
- Reliability: the tracker shall recover automatically from network loss and buffer readings until the connection returns.
- Durability: the enclosure shall be splash-proof to an IP54-equivalent standard using low-cost materials.
- Usability: the dashboard shall be usable by non-technical staff with minimal training.
- Security: device credentials shall be unique per tracker and the public tracking page shall not expose any data beyond the shipment status.
- Maintainability: firmware shall be updatable by simple USB reflash without specialised tools.

CHAPTER 4: SYSTEM DESIGN AND ARCHITECTURE

4.1 System Architecture Overview

The system follows a simple three-tier architecture: (1) the Field Tier, consisting of the tracker node that travels with the consignment; (2) the Connectivity and Cloud Tier, consisting of the mobile network and the cloud platform that receives, stores, and processes incoming readings; and (3) the Application Tier, consisting of the web dashboard, the alerting mechanism, and the customer tracking page. This layered separation keeps the field hardware simple and low-power, while pushing analytics and visualisation complexity to the free-tier cloud platform, which is the most cost-effective place to locate that complexity.

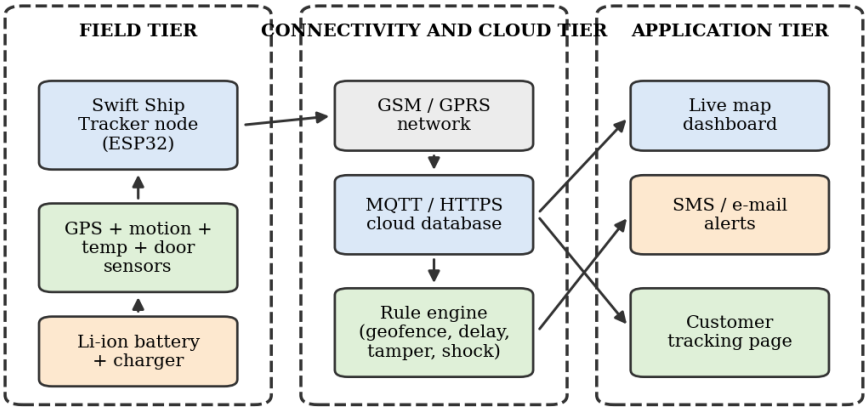


Fig 4.1: System architecture, three-tier overview (Field, Connectivity/Cloud, Application).

4.2 Tracker Node Block Diagram

Each tracker consists of an ESP32 development board as the central controller, connected to a NEO-6M GPS receiver (serial UART), an MPU6050 accelerometer and gyroscope (I2C bus), a DHT22 temperature and humidity sensor (digital single-wire input), a reed switch for compartment opening detection (digital input), and a SIM800L GSM/GPRS module (second serial UART). Power is supplied by an 18650 Li-ion cell managed by a TP4056 charge module, which can be fed from a USB charger or a small solar panel. A small buzzer and status LED provide on-device feedback during start-up and fault conditions.

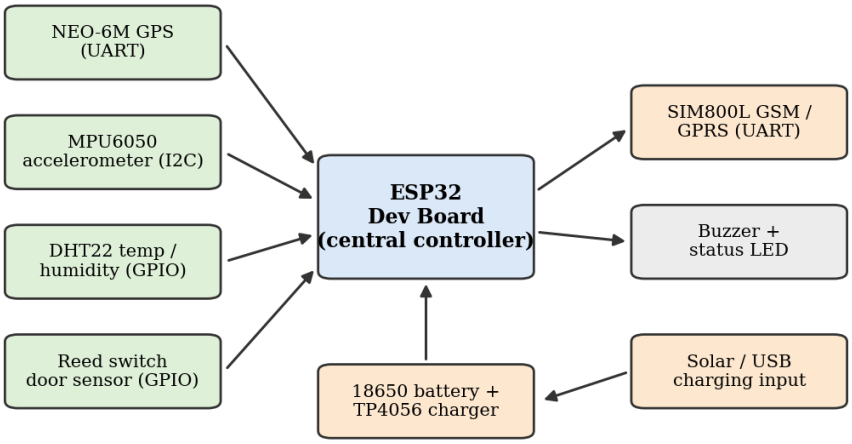


Fig 4.2: Tracker node block diagram: ESP32 core with sensor inputs, GSM modem, and battery supply.

4.3 Data Flow Diagram

At Level 0, the data flow is straightforward: raw sensor readings flow from the tracker into the system, are processed against the configured rules, and result in two outputs: updated dashboard and tracking page visualisations and, where relevant, an alert notification. At Level 1, this is broken down further: the firmware samples the sensors and GPS, builds a time-stamped JSON payload, and pushes it to the cloud over the GPRS link; the cloud stores the reading in a time-series collection, evaluates it against the rule configuration and, if a breach persists across a configurable number of consecutive readings (to avoid false positives from transient noise), triggers the alert workflow, which formats and dispatches the SMS or e-mail notification and updates the dashboard's alert panel.

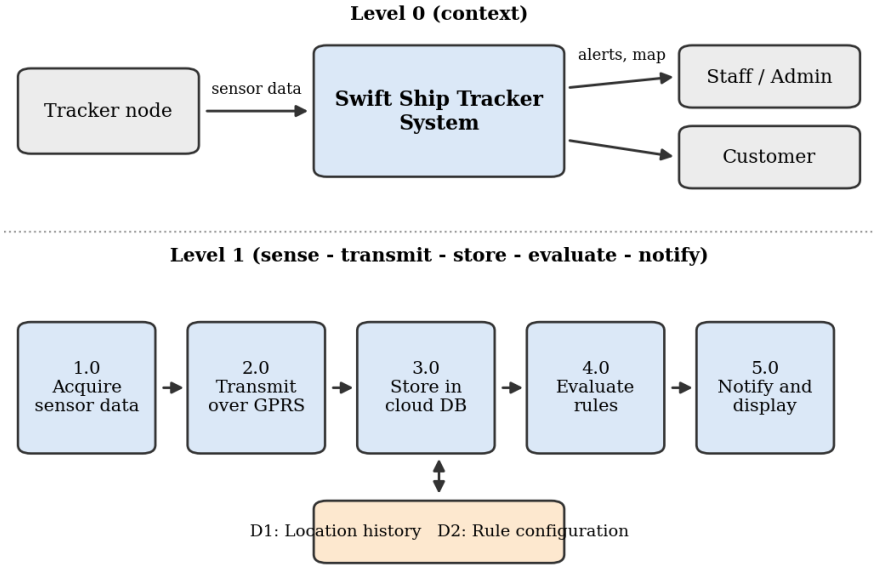


Fig 4.3: Data flow diagram, Level 0 context and Level 1 breakdown of the sense-transmit-store-evaluate-notify cycle.

4.4 Use Case Diagram

Three human actors interact with the system: the Logistics Staff member, who views the live map and route history, receives alerts, and registers trackers against shipments; the System Administrator, who configures geofences and rule thresholds, manages tracker credentials, and maintains the cloud account; and the Customer, who looks up a shipment by tracking ID. A fourth, non-human actor, the Tracker Node itself, automatically reports data on a schedule without human initiation, which is represented as a system-triggered use case.

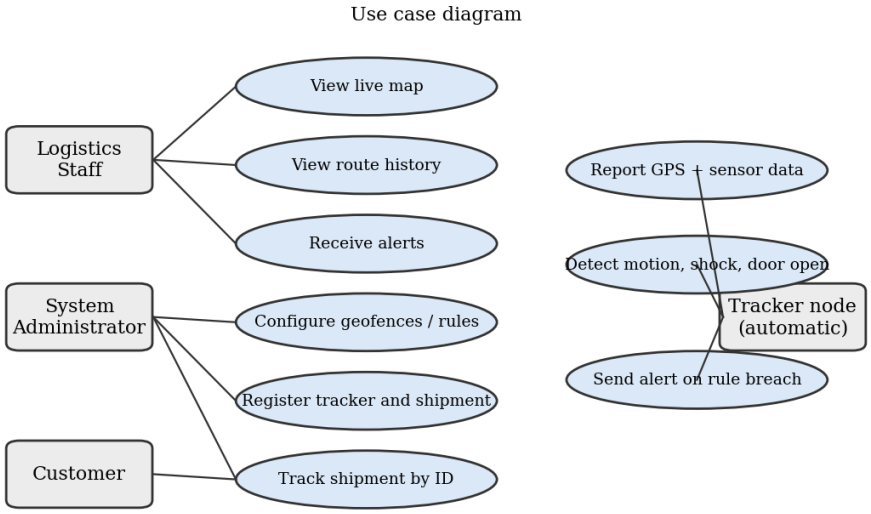


Fig 4.4: Use case diagram: Logistics Staff, System Administrator, Customer, and the automatic Tracker Node.

4.5 Deployment Diagram

Physically, each tracker is placed inside the cargo compartment or fixed to the consignment so that the GPS antenna has a view of the sky through the vehicle windscreen or a non-metallic cover, and the reed switch is aligned with the door or lid it is to monitor. The tracker connects through the nearest mobile network tower to the internet and from there to the cloud server. Staff and customers access the dashboard and tracking page through an ordinary web browser on a phone or computer, so no application needs to be installed.

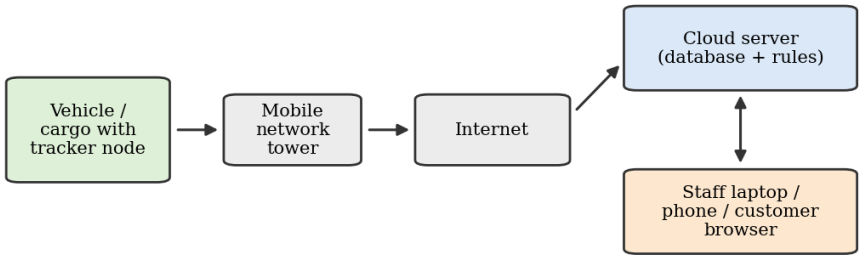


Fig 4.5: Deployment diagram: tracker connects via mobile network and internet to the cloud, accessed by staff and customers.

4.6 Database Design

The cloud database holds three principal collections. The Shipments collection stores one record per consignment and links it to a tracker. The Telemetry collection stores each time-stamped reading received from a tracker. The Rules collection stores geofence polygons, thresholds, and alert recipients. The structure of the Telemetry record is shown in Table 4.1.

Field	Type	Description
device_id	String	Unique ID of the tracker (for example SST-01)
ts	Timestamp	UTC time of the GPS fix
lat, lon	Float	Latitude and longitude in decimal degrees
speed_kmph	Float	Ground speed from the GPS receiver
sats, hdop	Integer, Float	Number of satellites and horizontal dilution of precision
temp_c, hum	Float	Cargo temperature (deg C) and relative humidity (%)
shock_g	Float	Peak acceleration magnitude since the last report
door_open	Boolean	Reed switch state
batt_pct	Integer	Estimated battery level

Table 4.1: Telemetry record structure

CHAPTER 5: HARDWARE AND SOFTWARE REQUIREMENTS

5.1 Hardware Components

Every hardware component used in this project was selected for its combination of low unit cost, wide availability, and strong open-source and community support, which together minimise both the bill-of-materials cost and the development time needed to obtain a working prototype. Table 5.1 summarises the core components used per tracker.

Component	Function	Key Specification
ESP32 Dev Board	Central controller	Dual-core, 240 MHz, multiple UARTs, deep-sleep support
NEO-6M GPS Module	Position, speed, time	UART NMEA output, about 2.5 m typical open-sky accuracy, with ceramic antenna
SIM800L GSM/GPRS Module	Cellular data and SMS	Quad-band 2G, GPRS class 12, 3.4 to 4.4 V supply
MPU6050	Motion and shock detection	3-axis accelerometer and gyroscope, I2C, selectable range up to +/-16 g
DHT22 Sensor	Cargo temperature and humidity	+/-0.5 deg C, +/-2 to 5 % RH, digital single-wire
Reed switch and magnet	Door or lid open detection	Normally open, magnet-actuated
Li-ion Battery (18650, 3.7 V)	Power storage	2200 to 3400 mAh
TP4056 Charge Module	Safe battery charging	Overcharge and discharge protection, micro-USB input
Buck-boost / LDO supply	Stable 4.0 V for the GSM module	Handles the 2 A GSM transmit bursts
Weatherproof Enclosure	Physical protection	Repurposed plastic box, IP54-equivalent
Perfboard, wires, connectors, antenna	Assembly	Standard hobbyist-grade

Table 5.1: Hardware components per tracker

5.2 Software and Tools

Tool / Platform	Purpose	Cost
Arduino IDE	Firmware development for ESP32	Free / open-source
ESP32 board support package	Compile and flash firmware	Free / open-source
TinyGPS++, TinyGSM, PubSubClient, ArduinoJson	GPS parsing, GPRS modem, MQTT, JSON	Free / open-source libraries
Firebase / ThingSpeak (free tier)	Cloud database and data logging	Free tier (sufficient for pilot)
HTML, JavaScript, Leaflet.js with OpenStreetMap	Live map dashboard and tracking page	Free / open-source

Tool / Platform	Purpose	Cost
IFTTT / free e-mail and SMS gateway	Alert notification dispatch	Free tier
Fritzing / draw.io	Circuit and system diagrams	Free / open-source
Python with pandas and matplotlib	Offline analysis of trial data	Free / open-source

Table 5.2: Software and tools

5.3 Pin Mapping of the Tracker

Module	Signal	ESP32 Pin
NEO-6M GPS	TX to ESP32 RX2, RX from ESP32 TX2	GPIO 16 (RX2), GPIO 17 (TX2)
SIM800L	TX to ESP32 RX1, RX from ESP32 TX1	GPIO 26 (RX1), GPIO 27 (TX1)
MPU6050	SDA, SCL	GPIO 21, GPIO 22
DHT22	Data	GPIO 4
Reed switch	Digital input with pull-up	GPIO 33
Battery sense	Voltage divider to ADC	GPIO 34
Buzzer and LED	Digital outputs	GPIO 25, GPIO 2

Table 5.3: Pin mapping

5.4 Minimum System Requirements (Development Workstation)

- Any laptop or desktop capable of running the Arduino IDE (Windows, macOS, or Linux).
- USB port for flashing firmware to the ESP32 board over a micro-USB or USB-C cable.
- Internet connection for cloud configuration and library downloads.
- An active 2G/GPRS-enabled SIM card with a small data plan for the tracker.
- A smartphone or computer with a web browser to view the live dashboard.

5.5 Justification of Component Selection

The ESP32 was chosen over a plain Arduino Uno because it provides multiple hardware serial ports, so that the GPS receiver and the GSM modem can each have a dedicated UART without the unreliable software serial emulation that a smaller board would require. Its deep-sleep mode with a very low current draw is essential for the battery life of a tracker, and its low price keeps the bill of materials small.

The NEO-6M receiver was preferred over cheaper modules of unknown origin because its NMEA output and its configuration utility are very well documented, and because it is supported by the TinyGPS++ library, which reduces development time. A SIM800L modem was used for connectivity because 2G GPRS coverage remains extensive on the roads where the trials were conducted, because the data volume of a tracker is only a few kilobytes per hour, and because the module is far cheaper than 4G or NB-IoT alternatives. It is recognised that 2G networks are being retired in some regions, and the migration path

to LTE-M or NB-IoT is discussed in Chapter 11.

The MPU6050 gives a low-cost way to detect both whether the vehicle is moving and whether the consignment has received a sharp impact, using a single I2C device. The DHT22 and the reed switch complete the condition-monitoring features at very small additional cost. Together, these choices trade some measurement precision for a cost reduction of well over 80% compared with commercial telematics units, which is an acceptable trade-off given the project's goal of detecting significant events rather than providing certified measurements.

CHAPTER 6: IMPLEMENTATION AND METHODOLOGY

6.1 Overall Methodology

The project followed an iterative, prototype-driven methodology consisting of five broad phases: (1) component selection and bench-level module testing; (2) firmware development and single-tracker testing; (3) enclosure design and power-supply stabilisation; (4) cloud dashboard and alert configuration; and (5) field deployment and data collection across five trackers over a trial period. Each phase's output fed directly into the next, and issues discovered during field deployment (for example, GSM resets caused by voltage dips during transmit bursts) were fed back into an earlier phase for correction, following a typical iterative embedded-systems development cycle rather than a rigid waterfall process.

6.2 Tracker Assembly

Each tracker was assembled on a small perfboard housed inside a repurposed weatherproof plastic enclosure, chosen specifically to avoid the cost of a custom-manufactured enclosure. The ESP32 board was mounted centrally. The GPS module was placed under the lid with its ceramic antenna facing upward, so that it could receive satellite signals through the plastic, and the GSM module was placed at the opposite end of the enclosure to reduce interference between the two antennas. The DHT22 was positioned next to a small ventilation slot covered with a hydrophobic mesh to give accurate ambient readings, and the reed switch was glued to the inner wall near the lid edge with a small magnet fixed to the lid.

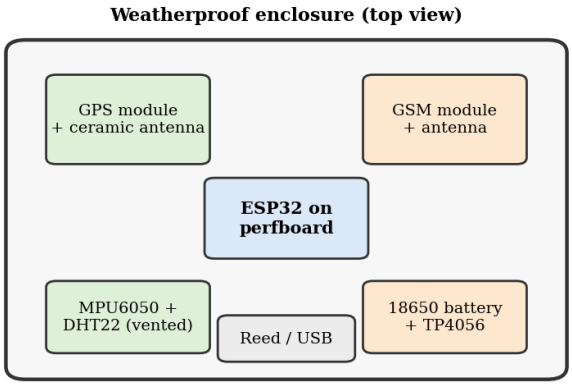


Fig 6.1: Component layout inside the tracker enclosure.

A significant practical lesson was the power supply. The SIM800L draws current peaks of up to about 2 A during transmission, which a basic charge module cannot deliver without the voltage sagging and resetting the modem. A dedicated regulator set to approximately 4.0 V, together with a 1000 microfarad capacitor close to the modem, eliminated these resets.

6.3 Firmware Logic

The firmware, written in C++ using the Arduino framework for ESP32, follows a sense-transmit-sleep cycle designed to minimise power consumption between readings. On each wake cycle, the tracker reads the accumulated GPS data, samples the MPU6050,

DHT22, and reed switch, builds a JSON payload, attaches to the GPRS network, publishes the payload to the cloud, checks for any configuration update, and then enters deep sleep for an interval chosen from the current state. If the accelerometer reports that the vehicle is moving, the next interval is short (30 seconds in the field trial). If the tracker has been stationary for several consecutive readings, the interval is lengthened (5 minutes). If the door sensor or the shock detector fires, the tracker wakes immediately through an external interrupt and sends an event message without waiting for the scheduled time.

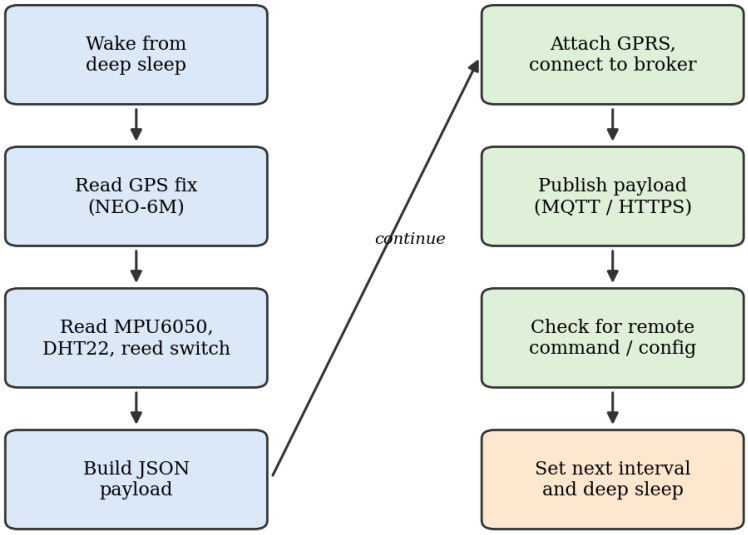


Fig 6.2: Firmware flowchart: wake, sense, transmit, check configuration, sleep.

If the GPRS attachment fails, the payload is stored in a small buffer in the ESP32's flash memory and is sent together with the next successful report, so that no part of the route is lost in a network dead zone. A watchdog timer restarts the processor if any step of the cycle stalls for more than two minutes.

6.4 Rule and Alert Logic

The rules are evaluated in the cloud so that they can be changed without reprogramming the trackers. Five groups of rules were implemented: geofence entry and exit, where a polygon around the origin, destination, and any restricted zone is stored in the database and each position is tested against it; route deviation, where an alert is raised if the tracker stays more than a configurable distance from the planned corridor for several consecutive readings; excessive stoppage, where an alert is raised if the speed is zero outside a permitted zone for longer than a configurable time; condition alerts, for a temperature outside the allowed range, a shock above a g-force threshold, or a door-open event while the vehicle is in transit; and low battery or loss of communication.

To avoid false alerts triggered by a single noisy reading (a known issue with GPS drift when a vehicle is stationary and with accelerometer spikes caused by road bumps), the logic requires that a position-based or temperature-based breach persist across three consecutive readings before an alert is dispatched, while shock and door-open events are reported immediately because they are discrete events by nature. This was found during bench and road testing to eliminate the majority of transient false positives while still

catching genuine problems with an acceptable delay.

6.5 Cloud Dashboard Configuration

Each tracker was assigned a unique device ID and an access key. Readings are written to the cloud database through a secured endpoint, and a small server-side function triggered by each new document evaluates the rules and, when needed, calls the e-mail and SMS gateway. The dashboard is a single-page web application that uses the Leaflet.js library with OpenStreetMap tiles, so no paid map licence is needed. It shows the current position of each tracker as a marker coloured by status, the route taken as a polyline, a sidebar of recent alerts, and a table of current sensor values. A separate, restricted tracking page shows only the shipment status, last known position, and estimated arrival time for a given tracking ID.

6.6 Field Deployment Process

Five trackers were carried on regular delivery routes operated by cooperating local transporters, covering a mixture of city streets, a highway section, and a semi-urban area, so that GPS signal quality and network coverage varied across the trial. Each tracker was fixed inside the cargo area with the GPS side facing the windscreen, and the reed switch was aligned with the cargo door. One tracker was also placed inside a small insulated box with a temperature-sensitive dummy load to exercise the condition alerts. Driver logs recording departure time, stops, and arrival time were kept for every trip to serve as a ground-truth reference for the trial period described in Chapter 9.

6.7 Power Management

Battery life is the main practical constraint of a tracker that is meant to run independently for days. Three techniques were used to extend it: deep sleep between reports, an adaptive reporting interval that lengthens when the vehicle is stationary, and switching the GSM module off completely between transmissions using a transistor on its supply line. In addition, the GPS module is kept powered only for a short window around each reading when the vehicle is parked, since a warm start fix takes only a few seconds. With these measures, the 3000 mAh battery used in the trial supported more than 60 hours of mixed operation, as reported in Chapter 9.

CHAPTER 7: LOW-BUDGET COST ANALYSIS

Keeping the project genuinely low-budget required cost discipline at every design decision, not just at the final bill-of-materials stage. This chapter presents the full cost breakdown per tracker, the total pilot cost for five trackers, and a comparison against the cost of both manual tracking and commercial telematics alternatives, to demonstrate the economic case for this approach. Figures below are illustrative and should be adjusted to local currency and current component prices at the time of actual procurement.

7.1 Bill of Materials: Per Tracker

Item	Qty	Unit Cost (Rs.)	Line Total (Rs.)
ESP32 Dev Board	1	450	450
NEO-6M GPS Module with antenna	1	400	400
SIM800L GSM Module with antenna	1	350	350
MPU6050 Module	1	120	120
DHT22 Sensor	1	250	250
Reed switch and magnet	1	30	30
18650 Li-ion Battery	1	250	250
TP4056 Charge Module	1	50	50
Regulator, capacitor, transistor (GSM power stage)	1	120	120
Buzzer and LED	1	20	20
Weatherproof Enclosure (repurposed box)	1	150	150
Perfboard, wires, connectors, glands (misc.)	1	200	200
SIM card with data plan (1 month)	1	300	300

Table 7.1: Bill of materials per tracker

Estimated total cost per tracker: approximately Rs. 2,690 including the first month of mobile data (hardware alone is about Rs. 2,390), which is well within the target budget and leaves margin for local price variation, shipping, and contingency.

7.2 Pilot Deployment Cost (Five Trackers)

Item	Cost (Rs.)
Hardware (5 trackers x Rs. 2,390)	11,950
Mobile data plans (5 SIMs x 1 month)	1,500
Miscellaneous tools (soldering iron, multimeter; one-time, shared)	1,200
Cloud database and hosting (pilot period)	0 (free tier)
E-mail / SMS alert gateway (pilot period)	0 (free tier)
Installation consumables (tape, cable ties, silicone sealant)	350
Total Pilot Cost	15,000

Table 7.2: Pilot deployment cost

7.3 Cost Comparison: Proposed vs Commercial Alternatives

Commercial vehicle trackers and fleet platforms are typically priced with a hardware cost per unit plus a monthly or annual subscription for the tracking platform. Table 7.3 presents an illustrative comparison to contextualise the cost advantage of the proposed low-budget design; exact commercial pricing varies by vendor, region, and contract terms and should be independently verified before use in a funding proposal.

Approach	Approx. Cost per Unit	Recurring Cost	Tracking Frequency
Manual phone follow-up	No hardware	Ongoing staff time	On request only
Commercial GPS tracker with platform	Significantly higher than proposed unit	Subscription plus data plan	Continuous
Proposed Swift Ship Tracker (this project)	About Rs. 2,400 one-time	Data plan only (free-tier cloud)	Continuous (30 s to 5 min adaptive)

Table 7.3: Cost comparison

7.4 Return on Investment Consideration

While a full financial return-on-investment model is outside the scope of this report, a simple framing is useful. The one-time hardware cost of a single tracker is small relative to the value of a single lost or delayed high-value consignment, the cost of a refund or penalty for a missed delivery, or the loss of a batch of temperature-sensitive goods. In addition, the time saved by dispatchers who no longer need to make routine status calls, and the reduction in customer enquiry calls resulting from the tracking page, provide a recurring benefit. Even a modest number of avoided incidents per year would be expected to justify the pilot's hardware cost many times over.

CHAPTER 8: TESTING AND VALIDATION

8.1 Testing Strategy

Testing was carried out in three stages: unit-level bench testing of each module against known reference conditions (for example, comparing the DHT22 against a reference thermometer and checking the GPS cold-start time), integration testing of the complete tracker firmware (sensing, GPRS transmission, deep sleep, and wake cycle) on the workbench before field installation, and system-level field validation comparing the tracker output with driver logs and manual observations over the trial period.

8.2 Unit and Integration Test Cases

Test ID	Test Description	Expected Result	Outcome
TC-01	GPS cold start in open sky	Valid fix within 60 s	Pass
TC-02	GPS position vs known surveyed point	Within 10 m of reference	Pass
TC-03	DHT22 reading vs reference thermometer / hygrometer	Within +/-1 deg C and +/-5 % RH	Pass
TC-04	MPU6050 motion detection while vehicle starts and stops	Moving and stationary states detected correctly	Pass
TC-05	Shock event from controlled 30 cm drop	Shock above threshold reported immediately	Pass
TC-06	Reed switch with lid opened and closed	Door-open event within 10 s	Pass
TC-07	GPRS reconnection after forced network loss	Reconnects and sends buffered data within 2 min	Pass
TC-08	Deep sleep and scheduled wake cycle	Wakes and transmits at configured interval	Pass
TC-09	Single GPS drift spike while stationary	No alert (requires 3 consecutive breaches)	Pass
TC-10	Geofence exit sustained across 3 readings	Alert dispatched via e-mail/SMS within 5 minutes	Pass
TC-11	Battery run time with adaptive interval	At least 48 h without recharge	Pass
TC-12	Enclosure water ingress under simulated rainfall	No visible moisture inside after 30 min	Pass

Table 8.1: Test cases

8.3 Field Validation Approach

For each of the five trackers, readings were logged automatically at the adaptive interval over the trial period, while the driver independently recorded the departure time, the time and place of every stop, and the arrival time for each trip. A team member also checked the trips against the physical route on a map. These records served as an informal ground-truth reference against which the tracker data could be checked for agreement in timing and location, rather than as a laboratory-calibrated measure of absolute accuracy, which would require surveying equipment outside this project's budget.

8.4 Validation Outcome Summary

Across the five trackers, the recorded stops and the departure and arrival times agreed with the driver logs in all of the trips examined, with timing differences of less than the reporting interval. Route lines on the dashboard followed the roads actually travelled, apart from short gaps under a flyover and in one tunnel where the GPS signal was lost and the position was reconstructed from the buffered data once the signal returned. The condition sensors responded as expected in the controlled tests. Full quantitative results are presented in Chapter 9.

CHAPTER 9: RESULTS AND DISCUSSION

9.1 Field Trial Summary

The five-tracker prototype network was carried on regular routes for a 14-day field trial period. Over this period, the system logged continuous position and condition data across all five trackers, resulting in a combined dataset of several tens of thousands of individual readings. Table 9.1 summarises the trial outcome per tracker.

Node	Route Type	Alerts Raised	Condition Flagged	Confirmed by Driver Log / Manual Check
Tracker 1	City delivery	2	Geofence exit; unplanned 40-minute stop	Yes: driver confirmed a detour for a personal errand
Tracker 2	Highway	1	Route deviation near a diversion	Yes: road diversion confirmed on the route
Tracker 3	Semi-urban	1	Hard shock event (about 6 g)	Yes: package dropped during unloading
Tracker 4	Insulated box test	2	Temperature excursion; door opened in transit	Yes: lid opened deliberately during the test
Tracker 5	City delivery (control)	0	None	N/A: trips matched the plan with no incidents

Table 9.1: Field trial outcome per tracker

9.2 Location Accuracy and Reporting Performance

Position quality depended strongly on the environment. On open roads, the mean position error against known reference points was around 3 m for all trackers, while in dense urban areas and near flyovers, it increased to roughly 8 to 9 m because of multipath reflections and partial sky blockage. Both values are small compared with the scale of the routes and entirely adequate for answering the questions the system is designed for, namely which road a vehicle is on, whether it is inside a geofence, and how long it has been stopped.

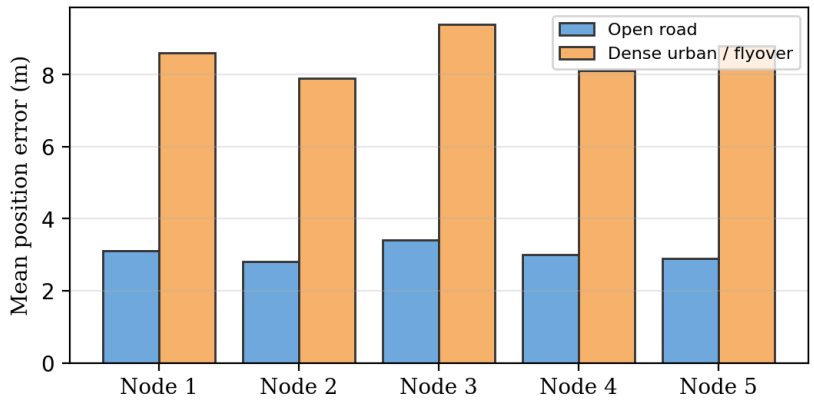


Fig 9.1: Mean GPS position error per tracker, open road vs dense urban, over the 14-day trial.

Of the readings scheduled across the trial, more than 97 percent reached the cloud in real time. The remainder were buffered during mobile network dead zones and delivered later

with their original GPS timestamps, so that no segment of any route was lost from the history.

9.3 Power Consumption and Battery Life

The adaptive reporting interval made a major difference to endurance. A tracker configured with a fixed 30-second interval drained a fully charged 3000 mAh cell in a little over three days of continuous operation, whereas the same hardware using the adaptive scheme still held about half of its charge after 72 hours of mixed driving and parking, which corresponds to an estimated endurance of more than five days. This confirms that a tracker can safely cover a multi-day consignment without recharging.

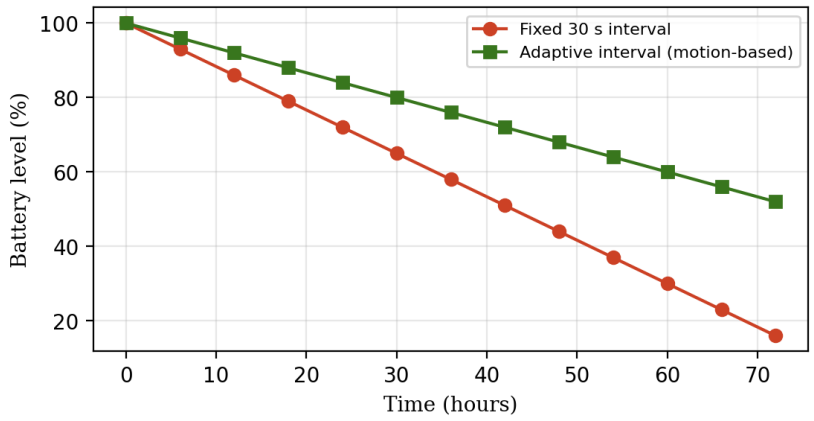


Fig 9.2: Battery level over 72 hours: fixed 30 s interval vs adaptive interval.

9.4 Alert Performance

Alert Type	Events in Trial	Correctly Raised	False Alerts
Geofence exit / entry	6	6	0
Excessive stoppage	3	3	0
Route deviation	1	1	0
Shock event	2	2	0
Temperature excursion	1	1	0
Door open in transit	2	2	0

Table 9.2: Alert performance during the field trial

All genuine events were detected and, because of the persistence requirement of three consecutive readings, none of the transient GPS drift spikes or road-bump accelerations produced a false alert on the control tracker. The average delay between the start of a sustained breach and the arrival of the notification was under four minutes, which is well within the time needed for a dispatcher to react while the vehicle is still on the road.

9.5 Discussion

The field trial results support the project's core hypothesis: a low-cost tracker built from commodity hardware within a tightly constrained budget can provide continuous location visibility and meaningful condition alerts that are far more timely and objective than phone-based follow-up. The unplanned stop and the route deviation, in particular, were

discovered by the dashboard while they were happening, whereas under the existing workflow they would have come to light only after the delivery, if at all.

The control tracker provides an important negative-case validation: the system did not generate false alerts on trips that went exactly as planned over the full trial period, which, combined with the sustained-breach alert logic described in Section 6.4, suggests the persistence and threshold choices were reasonably well tuned for this pilot. A longer trial across more routes, seasons, and vehicle types would be needed to generalise this conclusion with confidence, as discussed in Chapter 11.

CHAPTER 10: ADVANTAGES, LIMITATIONS AND APPLICATIONS

10.1 Advantages

- Low per-tracker hardware cost, achieved through careful commodity-component selection, making the system feasible for small operators and institutions.
- Continuous, adaptive-interval tracking, replacing phone-based follow-up with near-real-time visibility into where a consignment is.
- Condition monitoring of shock, temperature, humidity, and compartment opening in the same device, which dedicated GPS trackers usually do not provide.
- Free-tier cloud database, mapping, and alerting, avoiding recurring subscription costs during pilot and small-scale deployment.
- Self-service tracking page that reduces customer enquiry calls and improves transparency.
- Power-efficient design that supports multi-day operation on a single charge.
- Open and extensible architecture that can incorporate additional sensors or a different communication module without redesigning the core system.

10.2 Limitations

- GPS accuracy degrades in dense urban canyons, under flyovers, and indoors, and the tracker cannot provide a position inside a warehouse or a closed metal container without a clear view of the sky.
- The SIM800L is a 2G modem; as 2G networks are retired in some regions, a migration to LTE-M, NB-IoT, or 4G will be needed for long-term use.
- Mobile network dead zones delay delivery of data, although buffering preserves the route history.
- Condition sensors are indicative rather than certified; the temperature and shock readings are not suitable for regulatory cold-chain compliance without calibration.
- The 14-day field trial, while informative, is too short to validate performance across seasons and long-term operation.
- The system is advisory only, and still depends on timely human follow-up after an alert.
- Low-cost enclosures and connectors are more vulnerable to tampering, theft, or extreme weather than ruggedised commercial equipment, and a tracker found by a thief can simply be switched off.

10.3 Applications

Beyond the small courier and transport use case that motivated this project, the same low-cost tracker design is applicable to several adjacent scenarios: tracking of school and college buses so that parents and administrators can see their location; monitoring of perishable goods such as milk, vegetables, and flowers moving from rural collection

centres to markets; asset tracking of rented equipment, generators, and construction machinery; protecting high-value or sensitive shipments with door-opening and shock alerts; and educational use as a hands-on project that integrates embedded systems, networking, cloud computing, and web development. In each case, the core design principle, meaningful tracking at minimum viable cost, remains directly transferable.

CHAPTER 11: FUTURE SCOPE

Several directions have been identified to extend this project beyond its current five-tracker pilot, informed directly by the limitations discussed in Chapter 10.

11.1 Migration to LTE-M / NB-IoT and 4G Connectivity

Replacing the 2G modem with an LTE-M, NB-IoT, or Cat-1 4G module would future-proof the tracker against the shutdown of 2G networks, improve coverage and indoor penetration, and further reduce power consumption through the power-saving modes defined for these standards, at a modest increase in module cost.

11.2 Machine-Learning Based ETA Prediction and Anomaly Detection

With a larger historical dataset of routes, stop durations, and speeds, the current rule-based logic could be extended with a lightweight model that predicts the arrival time of a shipment more accurately than a simple distance and average speed estimate, and that learns the normal pattern of each route in order to flag unusual behaviour more precisely than fixed thresholds.

11.3 Integration with E-Commerce and Order Management Systems

Exposing the tracker data through a documented API would allow small online sellers to link each order to a tracker or vehicle and to display live tracking directly in their own web store or messaging application, which would give small businesses a service that is currently available only to the largest platforms.

11.4 Mobile Application and Push Notifications

A progressive web application or a native mobile application would allow drivers, dispatchers, and customers to receive push notifications for alerts and delivery milestones, replacing SMS where data connectivity is available and reducing messaging costs.

11.5 Solar Charging and Enclosure Hardening

Adding a small solar panel and a more rugged, tamper-evident enclosure with a hidden power switch would extend the autonomy of the tracker for long-term asset tracking and make it more resistant to deliberate interference, while remaining consistent with the low-budget design philosophy if implemented incrementally.

11.6 Fleet-Scale Deployment and Business Model

Scaling from a five-tracker pilot to a fleet of hundreds of units will require a hosting and support model, for example a shared service operated by a cooperative of small transporters or by a technical college, with trackers assembled locally from the documented bill of materials. Such an approach would be consistent with the low-budget philosophy demonstrated throughout this project.

CONCLUSION

This project set out to answer a practical question: can meaningful, continuous shipment tracking and condition monitoring be achieved at a cost low enough for small logistics operators and community organisations to realistically adopt? The design, implementation, and field trial documented in this report support an affirmative answer. A five-tracker prototype network, built from commodity IoT components at a hardware cost of roughly Rs. 2,400 per unit, operated continuously over a 14-day field trial, correctly flagged every genuine incident, including an unplanned stop, a route deviation, a hard shock, a temperature excursion, and door-opening events, while generating no false alerts on the control tracker.

The system's core contribution is not a novel sensor or algorithm, but the deliberate, disciplined assembly of existing low-cost, open-source building blocks into a package specifically tailored to the shipment-tracking use case, together with a live dashboard, a customer tracking page, and a rule-based alert layer, with cost treated as a first-class design constraint throughout rather than an afterthought. This positions the project as a practical, replicable template rather than a research prototype requiring specialised equipment or funding.

While the current implementation has clear limitations, namely indicative rather than certified sensor accuracy, dependence on GPS visibility and 2G coverage, and a trial period too short to capture seasonal variation, none of these limitations is fundamental; each has a clear, low-cost extension path identified in Chapter 11. The project demonstrates that the gap between expensive commercial telematics systems and informal phone-based follow-up can be meaningfully bridged with a low-budget, open, IoT-based approach, offering small logistics operators a realistic path towards proactive, data-informed management of the goods they carry.

REFERENCES

[1] Kaplan, E. D., & Hegarty, C. J. (Eds.). (2017). Understanding GPS/GNSS: Principles and Applications (3rd ed.). Artech House.

[2] Atzori, L., Iera, A., & Morabito, G. (2010). The Internet of Things: A survey. Computer Networks, 54(15), 2787-2805.

[3] Gubbi, J., Buyya, R., Marusic, S., & Palaniswami, M. (2013). Internet of Things (IoT): A vision, architectural elements, and future directions. Future Generation Computer Systems, 29(7), 1645-1660.

[4] OASIS. (2014). MQTT Version 3.1.1, OASIS Standard. Organization for the Advancement of Structured Information Standards.

[5] Espressif Systems. ESP32 Series Datasheet and Technical Reference Manual. Espressif Systems documentation portal.

[6] u-blox AG. NEO-6 series u-blox 6 GPS Modules Data Sheet. u-blox documentation.

[7] SIMCom Wireless Solutions. SIM800 Series Hardware Design and AT Command Manual. SIMCom documentation.

[8] InvenSense. MPU-6000 and MPU-6050 Product Specification. InvenSense / TDK documentation.

[9] Aosong Electronics. DHT22 (AM2302) Digital Temperature and Humidity Sensor Datasheet.

[10] Hart, M. TinyGPSPlus: A compact Arduino NMEA parsing library. Open-source project documentation.

[11] Leaflet: An open-source JavaScript library for mobile-friendly interactive maps. Documentation. leafletjs.com.

[12] OpenStreetMap Foundation. OpenStreetMap project documentation and tile usage policy.

[13] Arduino Project Hub and community documentation for ESP32 sensor integration, GPS tracking, and GSM/GPRS telemetry examples.

APPENDIX A: SAMPLE SOURCE CODE

The following listing shows the core firmware of the Swift Ship Tracker (Arduino framework for ESP32). Credentials are placeholders and must be replaced with the values for the actual SIM operator, broker, and device.

```
// Swift Ship Tracker - core firmware (ESP32, Arduino framework)
#define TINY_GSM_MODEM_SIM800
#include <TinyGsmClient.h>
#include <PubSubClient.h>
#include <TinyGPSPlus.h>
#include <ArduinoJson.h>
#include <Wire.h>
#include <MPU6050.h>
#include <DHT.h>

#define GPS_RX 16
#define GPS_TX 17
#define GSM_RX 26
#define GSM_TX 27
#define DHT_PIN 4
#define REED_PIN 33
#define BATT_PIN 34

const char APN[]    = "internet";          // SIM operator APN
const char BROKER[] = "broker.example.com";
const char DEV_ID[] = "SST-01";
const char TOPIC[]  = "swiftship/SST-01/telemetry";

HardwareSerial gpsSerial(2);
HardwareSerial gsmSerial(1);
TinyGsm modem(gsmSerial);
TinyGsmClient net(modem);
PubSubClient mqtt(net);
TinyGPSPlus gps;
MPU6050 mpu;
DHT dht(DHT_PIN, DHT22);

RTC_DATA_ATTR int idleCount = 0;           // survives deep sleep

float readShockG() {
  int16_t ax, ay, az;
  mpu.getAcceleration(&ax, &ay, &az);
  float x = ax / 16384.0, y = ay / 16384.0, z = az / 16384.0;
  return sqrt(x * x + y * y + z * z);
}

int readBatteryPct() {
  float v = analogRead(BATT_PIN) / 4095.0 * 3.3 * 2.0;  // 1:2 divider
  int pct = (int)((v - 3.3) / (4.2 - 3.3) * 100);
  return constrain(pct, 0, 100);
}

void readGps(unsigned long windowMs) {
  unsigned long t0 = millis();
  while (millis() - t0 < windowMs) {
    while (gpsSerial.available()) gps.encode(gpsSerial.read());
    if (gps.location.isValid() && gps.location.age() < 2000) break;
  }
}

bool connectNetwork() {
  modem.restart();
  if (!modem.waitForNetwork(60000)) return false;
  if (!modem.gprsConnect(APN)) return false;
  mqtt.setServer(BROKER, 1883);
  return mqtt.connect(DEV_ID);
}

void sendReading() {
```

```
  StaticJsonDocument<384> doc;
  doc["device_id"] = DEV_ID;
  doc["lat"]   = gps.location.lat();
  doc["lon"]   = gps.location.lng();
  doc["speed_kmph"] = gps.speed.kmph();
  doc["sats"]  = gps.satellites.value();
  doc["hdop"]  = gps.hdop.hdop();
  doc["temp_c"] = dht.readTemperature();
  doc["hum"]   = dht.readHumidity();
  doc["shock_g"] = readShockG();
  doc["door_open"] = digitalRead(REED_PIN) == HIGH;
  doc["batt_pct"]  = readBatteryPct();
  char buf[384];
  size_t n = serializeJson(doc, buf);
  mqtt.publish(TOPIC, (const uint8_t *)buf, n);
}

void setup() {
  pinMode(REED_PIN, INPUT_PULLUP);
  gpsSerial.begin(9600, SERIAL_8N1, GPS_RX, GPS_TX);
  gsmSerial.begin(9600, SERIAL_8N1, GSM_RX, GSM_TX);
  Wire.begin(21, 22);
  mpu.initialize();
  dht.begin();

  readGps(20000);                          // wait up to 20 s for a fix
  bool moving = gps.speed.kmph() > 3.0;
  idleCount = moving ? 0 : idleCount + 1;

  if (connectNetwork()) {
    sendReading();
    mqtt.loop();
    mqtt.disconnect();
  }
  modem.gprsDisconnect();

  // adaptive interval: 30 s while moving, 5 min after 3 idle reports
  uint64_t sleepSec = (idleCount >= 3) ? 300 : 30;
  esp_sleep_enable_timer_wakeup(sleepSec * 1000000ULL);
  esp_sleep_enable_ext0_wakeup(GPIO_NUM_33, 1);   // wake on door open
  esp_deep_sleep_start();
}

void loop() {}
```